

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

Noise and GPs

Families

Adaptive
shapes

The fin

Hats to
networks

Representing Functions and Fields

Finitely many numbers for a function, and what each choice loses

Krishna Kumar

University of Texas at Austin

krishnak@utexas.edu

Representing a function as a discrete list and a rule

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

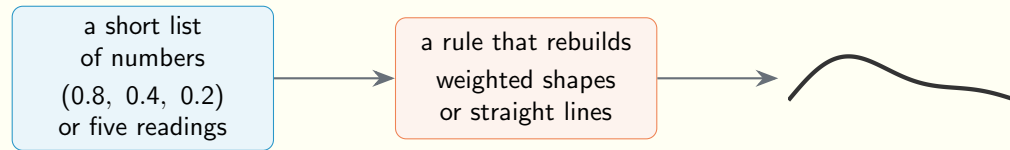
Noise and GPs

Families

Adaptive
shapes

The fin

Hats to
networks



How well can finitely many numbers stand in for a function, and what decides the accuracy?

The displacement of a string is a curve. A computer cannot store a curve, so every method in this course stores a short list of numbers and a rule that turns the list back into a curve, and the perceptron is one such choice.

How well can finitely many numbers stand in for a function, and what decides the accuracy?

Outline

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

Noise and GPs

Families

Adaptive
shapes

The fin

Hats to
networks

- 1 Readings at fixed stations
- 2 Coefficients on shapes
- 3 Norms and the meaning of error
- 4 Inner products, projection, and orthogonality
- 5 Changing the basis
- 6 Convergence, density, and completeness
- 7 Kernels and point evaluation
- 8 Noisy observations, ridge regression, and Gaussian conditioning
- 9 Approximating a family of functions
- 10 Adaptive representations
- 11 Choosing a representation for the cooling fin
- 12 From hats to a network

Sampling the displacement at fixed stations

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

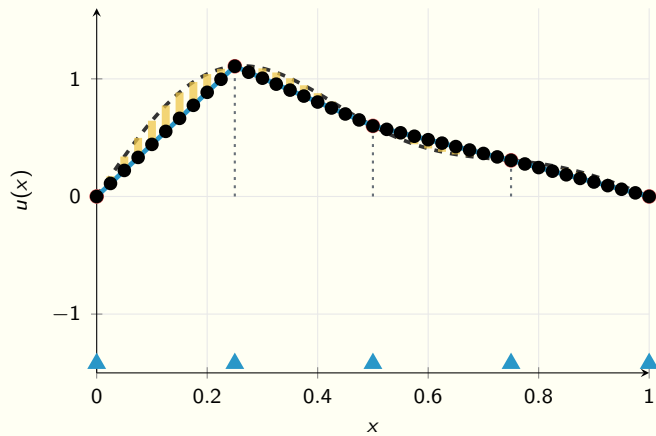
Noise and GPs

Families

Adaptive shapes

The fin

Hats to networks



five readings
0, 1.11, 0.60, 0.31, 0

worst miss, five sensors
0.22, or 20.1 percent

[Try it live](#)

Here is a taut string, pulled sideways. The whole curve is its displacement, and we want to store it in a computer with a handful of numbers. How?

The obvious move is to measure it. Put five sensors along the string, at $x = 0, \frac{1}{4}, \frac{1}{2}, \frac{3}{4}, 1$.

They give us five readings, 0, 1.11, 0.60, 0.31, 0. Everything between the sensors is unknown. So we guess. The simplest guess connects neighboring readings with straight lines, and we have a curve again.

Now look where the guess is wrong. The lines miss the bends between sensors, and the worst miss is 0.22, about a fifth of the largest displacement. Would more sensors fix this?

Convergence of piecewise-linear interpolation with the spacing

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

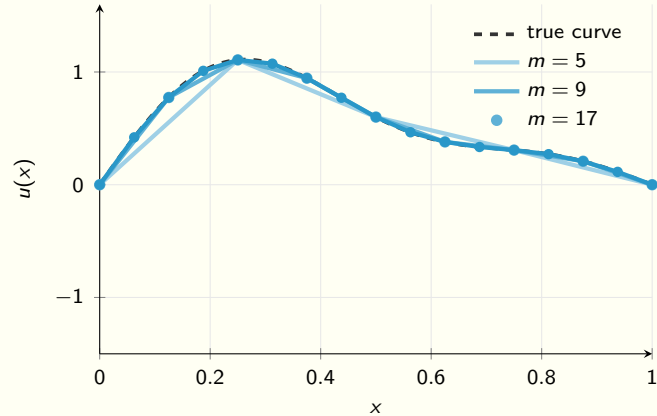
Noise and GPs

Families

Adaptive
shapes

The fin

Hats to
networks



$m = 5, h = 1/4$

20.1 percent

$m = 9, h = 1/8$

**6.3 percent, a factor
of 3.2**

$m = 17, h = 1/16$

**1.6 percent, a factor
of 4.0**

slope on log axes
-2.07

So we add sensors. Before adding any, guess how fast the miss should fall. Between two sensors a distance h apart, the curve bends away from a straight line by about $u''h^2/8$ at the middle, so halving h should cut the miss to a quarter.

Nine sensors give 6.3 percent, a factor of 3.2 and short of four. At five sensors one segment spans more than a third of the shortest wavelength in the curve, so the bend inside it is far from a parabola and the estimate $u''h^2/8$ is too crude. The estimate applies once h is small against that wavelength.

Seventeen sensors give 1.6 percent, a factor of 4.0, and from here on every halving divides the miss by four, a slope of -2.07 on logarithmic axes. We can draw these gaps only because we know the curve. What else do the readings alone hide?

Aliasing of a wave that vanishes at every station

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

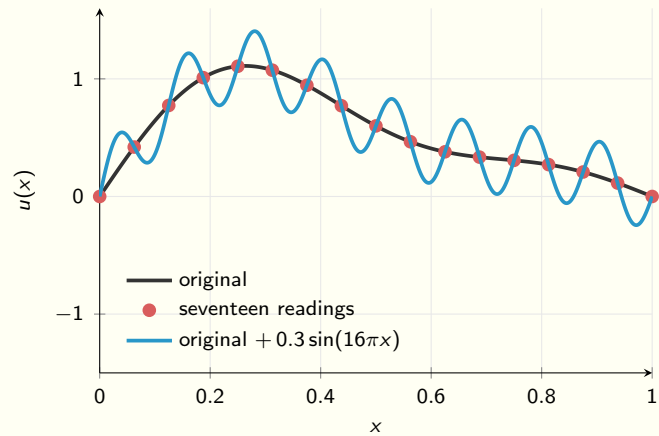
Noise and GPs

Families

Adaptive shapes

The fin

Hats to networks



change in the seventeen readings

0

change at each crest
0.3, about a quarter of the largest displacement

[Try it live](#)

Suppose all we have is these seventeen readings. Which curves could have produced them?

Take one. Add the wave $0.3 \sin(16\pi x)$ to the displacement. At every station $16\pi x_i$ is a multiple of π , so the wave is zero there and not one reading changes, while the curve moves by 0.3 at each crest, about a quarter of the largest displacement.

Two functions that agree at every station are aliases on that grid, and the readings cannot tell them apart. Could a few numbers describe the whole curve at once, instead of one place each?

Grid dependence of sampled representations

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

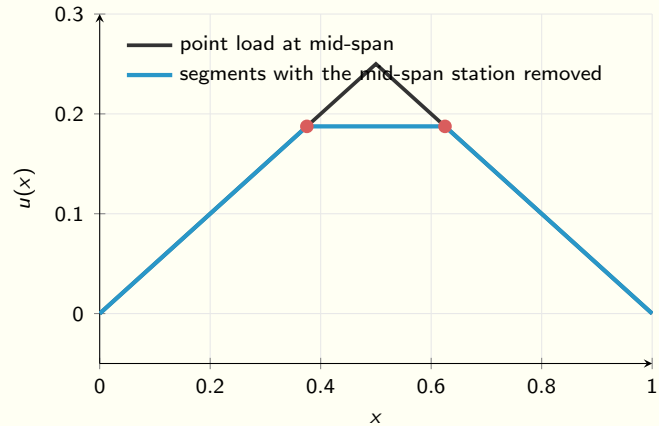
Noise and GPs

Families

Adaptive shapes

The fin

Hats to networks



corner, mid-span sensor removed

worst miss 0 to 12.5 percent

smooth displacement, same sensor removed

1.6 to 1.7 percent

A reading also means something only together with the place it was taken. Two lists from different grids cannot be compared entry by entry, and a sensor that fails leaves a hole for the rule to fill.

Press on the string at mid-span, so it bends into two straight segments with a corner. Remove the sensor on the corner and the straight lines cut the peak off, so the worst miss jumps from zero to 12.5 percent. Remove the same sensor from the smooth displacement and the miss barely moves, from 1.6 to 1.7 percent. Is there a description in which no number belongs to one place?

Representing a function with coefficients on sine modes

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

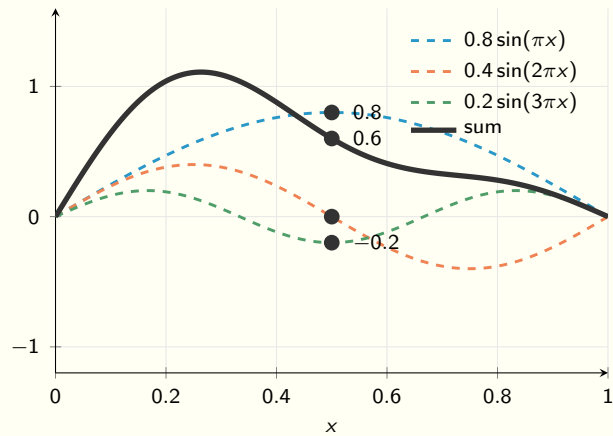
Noise and GPs

Families

Adaptive
shapes

The fin

Hats to
networks



the whole curve
three numbers,
 $(0.8, 0.4, 0.2)$

check at mid-span
 $u(\frac{1}{2}) = 0.8 - 0.2 = 0.6$

[Try it live](#)

Here is a different idea. Instead of asking where the string is, ask what it is made of. The string vibrates in sine modes, and the plot shows the first one weighted by 0.8.

Add the second mode, weighted by 0.4. Add the third, weighted by 0.2.

Add the three together and the displacement of the string appears. Three numbers, $(0.8, 0.4, 0.2)$, give the whole curve, with no grid anywhere.

Check it at mid-span. The second mode vanishes there, the first and third read 1 and -1 , so $u(\frac{1}{2}) = 0.8 - 0.2 = 0.6$. And since every sine vanishes at both ends, every curve we can build this way is pinned.

The coefficient list as coordinates in a vector space

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

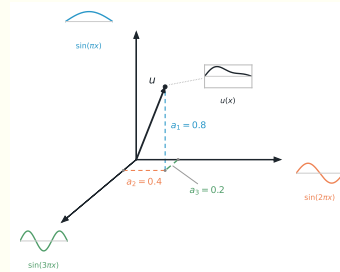
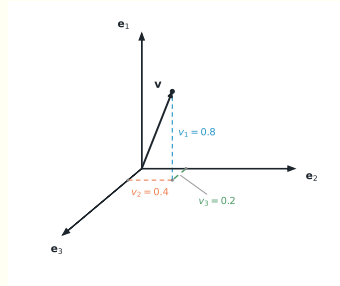
Noise and GPs

Families

Adaptive
shapes

The fin

Hats to
networks



$$u + v = \sum_{i=1}^3 (a_i + b_i) \phi_i, \quad \alpha u = \sum_{i=1}^3 (\alpha a_i) \phi_i$$

Look at what the three numbers are. On the left, $(0.8, 0.4, 0.2)$ is an arrow with three components. On the right, the same three numbers are the displacement.

Add two displacements and their coefficients add, scale one and its coefficients scale, so the family is closed, the three modes are a basis, and a displacement is a point in a three-dimensional space of functions.

Approximating a function outside the span

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

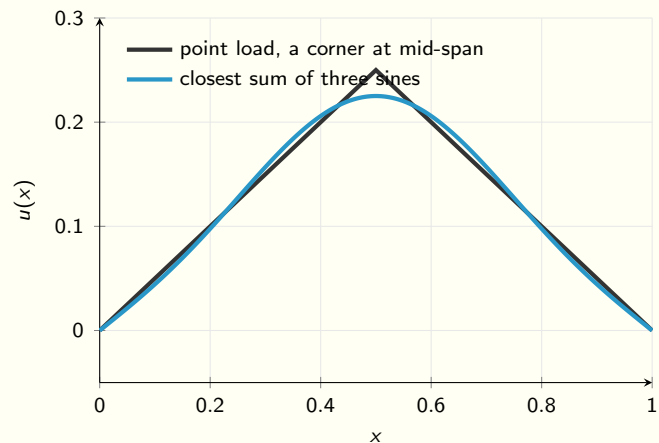
Noise and GPs

Families

Adaptive
shapes

The fin

Hats to
networks



closest three sines,
root-mean-square miss
4.8 percent

So far every curve was smooth, because every sine is smooth. Now press on the string at one point. The point load bends it into two straight segments with a corner at mid-span.

No sum of three sines has a corner, so the best we can do is the closest sum, and it misses by 4.8 percent in the root-mean-square sense. A fixed family represents its own members without error and approximates everything outside it, and the sensors did the same. Before we can say which sum is closest, we have to say how we measure an error.

Measuring error in the sup, L^2 , and H^1 norms

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

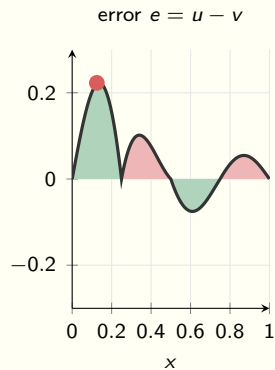
Noise and GPs

Families

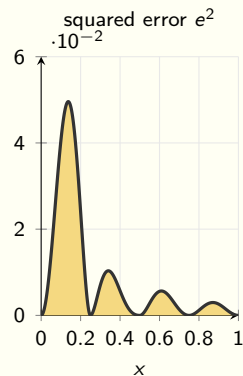
Adaptive
shapes

The fin

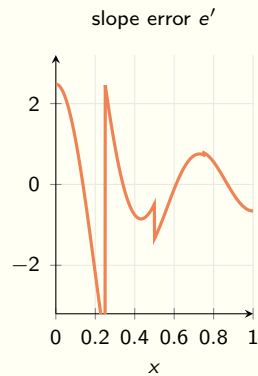
Hats to
networks



largest gap, sup norm
0.22



root of the area, L^2 norm
0.093



root-mean-square slope,
 H^1 seminorm
1.20

[Try it live](#)

The error of the five-sensor guess is $e = u - v$. How large is it? That depends on what we care about. If it is clearance under the string, only the worst gap matters, 0.22, and that is the sup norm.

If it is the overall misfit, all the gaps count at once. Their average fails, because positive and negative gaps cancel, so square first, integrate, and take the root. That is the L^2 norm, 0.093.

If it is the elastic energy, the values are the wrong thing to look at, since the energy is half the integral of the squared slope. The root of the integrated squared slope error is the H^1 seminorm, 1.20, and half its square is the energy of the error, 0.72. In relative terms the values are off by 14 percent and the slopes by 42.

Disagreement between the sup, L^2 , and H^1 norms

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

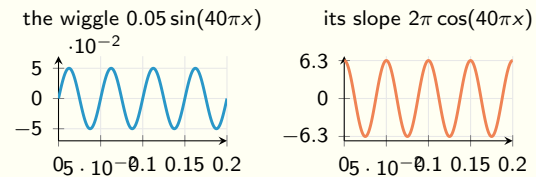
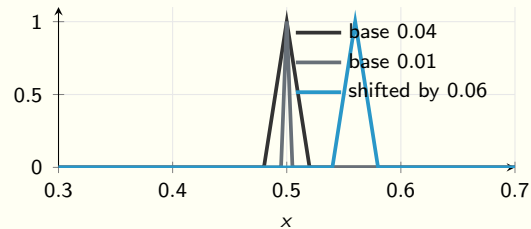
Noise and GPs

Families

Adaptive
shapes

The fin

Hats to
networks



spike, base 0.04
sup 1, L^2 0.115,
energy 50

spike, base 0.01
sup 1, L^2 0.058,
energy 200

L^2 distance, spike to
shifted spike
 $\sqrt{2} \times 0.115 = 0.163$

wiggly $0.05 \sin(40\pi x)$
sup 0.05, L^2 0.035,
 H^1 4.44

[Try it live](#)

Can the three disagree by a lot? Take a spike of height one on a base of width 0.04. Its sup norm is one. Its L^2 norm is $\sqrt{b/3} = 0.115$, and its slopes are $\pm 2/b$, so its elastic energy is $2/b = 50$.

Quarter the base. The sup norm stays at one, the L^2 norm halves to 0.058, and the energy climbs to 200. No multiple of the L^2 norm bounds the sup norm, and a displacement of finite energy cannot contain a spike of zero width.

Shift the spike by one and a half base widths, so it overlaps the original nowhere. The L^2 distance between the two is $\sqrt{2} \times 0.115 = 0.163$, because the L^2 distance compares the two point by point and they never overlap. A small fast wiggle, $0.05 \sin(40\pi x)$, is small in the sup and L^2 norms, 0.05 and 0.035, and its slope swings to ± 6.3 , so its H^1 seminorm is 4.44.

Definition of a norm and a seminorm

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

Noise and GPs

Families

Adaptive
shapes

The fin

Hats to
networks

Norm

$$\|u\| = 0 \text{ only for } u = 0, \quad \|\alpha u\| = |\alpha| \|u\|, \quad \|u + v\| \leq \|u\| + \|v\|.$$

Seminorm

$$|c|_{H^1} = \left(\int_0^1 (c')^2 dx \right)^{1/2} = 0 \quad \text{for every constant } c, \quad \text{and on the pinned string the only constant is 0.}$$

We have been calling three different rules a size. What do they share? Each gives the zero function size zero and nothing else size zero, each scales with the function, and each gives a sum at most the sum of the sizes. A rule with these three properties is a norm.

The slope rule misses the first property, since a constant function has zero slope, so on its own it is a seminorm. On the pinned string the only constant allowed is zero, and there the slope rule is a norm after all.

So the phrase closest member of a family means nothing until we name the norm. Name L^2 , and ask which sum of three sines is closest to the shape of the string under a uniform load, the parabola $x(1-x)$.

The inner product as signed overlap

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

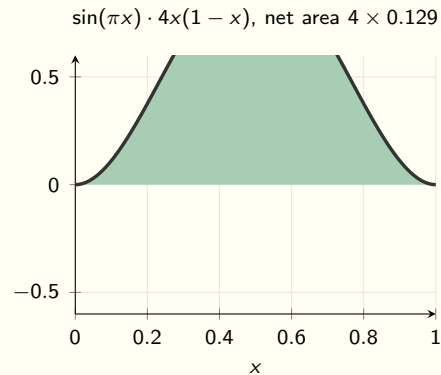
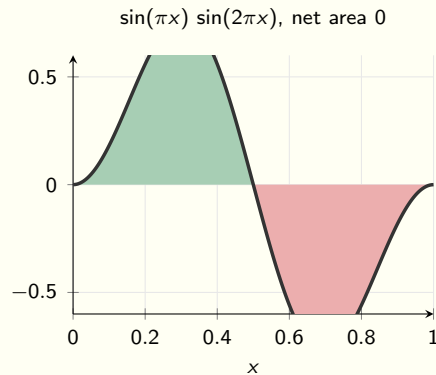
Noise and GPs

Families

Adaptive shapes

The fin

Hats to networks



$$\langle f, g \rangle = \int_0^1 f(x) g(x) dx$$

The uniform load $f = 2$ bends the string into the parabola $x(1-x)$, and no sum of three sines equals it. In ordinary geometry a perpendicular dropped from the point onto the plane finds the closest point, and a perpendicular needs a dot product. For functions the dot product is the integral of the product, $\langle f, g \rangle = \int_0^1 fg dx$, the net signed area under $f(x)g(x)$.

Try $\sin(\pi x)$ against $\sin(2\pi x)$. One is even about mid-span and the other odd, so the two halves cancel and the modes are orthogonal.

Now the parabola against $\sin(\pi x)$. The two never oppose in sign, so the area is positive, 0.129, which is $4/\pi^3$, and integrating $-u'' = 2$ by parts twice gives it without integrating the parabola at all.

Minimizing squared error via orthogonal projection

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

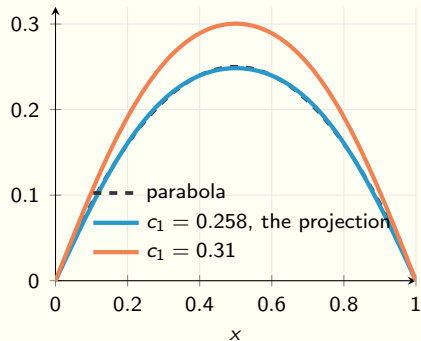
Noise and GPs

Families

Adaptive shapes

The fin

Hats to networks



$c_1 = 8/\pi^3 = 0.258$
error 0.87 percent

$c_1 = 0.31$, twenty percent high
error norm 23 times larger

[Try it live](#)

Fix $c_2 = 0$ and $c_3 = 8/(27\pi^3)$, and slide c_1 . The squared error is a quadratic in c_1 , and its slope is $-2\langle u - v, \sin(\pi x) \rangle$.

The bottom of the bowl sits where that inner product vanishes, where the error is orthogonal to the mode. That gives $c_1 = 2\langle u, \sin(\pi x) \rangle = 8/\pi^3 = 0.258$, one division per coefficient because the modes are orthogonal to each other, and an error of 0.87 percent.

Push c_1 twenty percent too high, to 0.31. To the eye the curve looks as good, and its error norm is 23 times larger, since the squared error rises from 2.5×10^{-6} to 1.35×10^{-3} .

Uniqueness of the projection and the Pythagorean identity

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

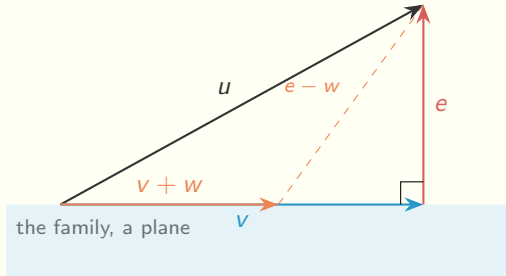
Noise and GPs

Families

Adaptive
shapes

The fin

Hats to
networks



$$\|e - w\|^2 = \|e\|^2 + \|w\|^2$$
$$\|u\|^2 = \|v\|^2 + \|e\|^2, \quad 0.033333 = 0.033331 + 2.5 \times 10^{-6}$$

Why is the perpendicular the best we can do? Write $u = v + e$ with e orthogonal to every mode. Move to any other combination $v + w$. The new error is $e - w$, and because e is orthogonal to w , $\|e - w\|^2 = \|e\|^2 + \|w\|^2$. Every move away from v adds its own squared length to the error, so v is the closest combination and the only one.

Take $w = -v$ and we get Pythagoras, $\|u\|^2 = \|v\|^2 + \|e\|^2$, which predicts the error before we draw anything. $\|u\|^2 = 1/30 = 0.033333$ and $\|v\|^2 = 0.033331$, so $\|e\|^2 = 2.5 \times 10^{-6}$, and the 0.87 percent follows.

The projection theorem

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

Noise and GPs

Families

Adaptive
shapes

The fin

Hats to
networks

Projection theorem

In a space with an inner product, a finite-dimensional family has one closest member to each target, and that member is the one whose error is orthogonal to the whole family.

$$\langle u-v, \phi_j \rangle = 0 \quad \text{for every shape } \phi_j, \quad c_k = \frac{\langle u, \phi_k \rangle}{\langle \phi_k, \phi_k \rangle} \text{ when the shapes are orthogonal.}$$

Orthogonality gives one linear equation per shape, so the closest member is computed and never searched for, and when the shapes are orthogonal to one another each coefficient is a single division. The Fourier coefficients of a function are these divisions, one per mode.

For a family with infinitely many shapes the theorem needs two more hypotheses, a complete space and a family that contains its own limits. Is the inner product itself fixed, or a choice?

Projection in the L^2 and energy inner products

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

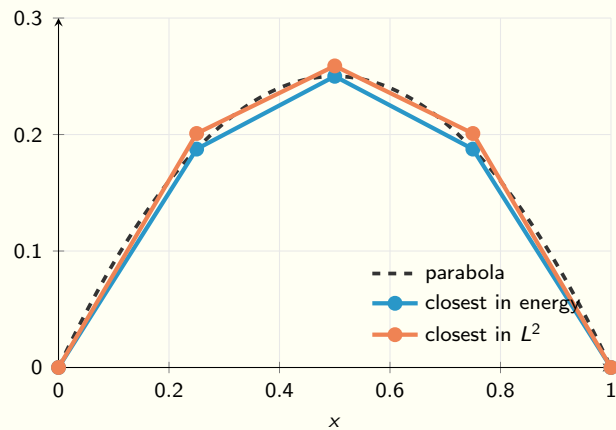
Noise and GPs

Families

Adaptive shapes

The fin

Hats to networks



closest in energy, heights
0.1875, 0.25, 0.1875

closest in L^2 , heights
0.2009, 0.2589, 0.2009

L^2 error, L^2 against energy
3.3 **against** 6.3
percent

H^1 error, L^2 against energy
26.0 **against** 25.0
percent

The inner product was our choice. The energy inner product $\int f'g' dx$ is another, and it gives another closest member. Keep the parabola and take a new family, straight segments with breaks at 0.25, 0.5, 0.75. In energy the closest member passes through the parabola's own values, 0.1875, 0.25, 0.1875, and it is the straight-segment guess from the first slides, because its slope error integrates to zero across every segment.

In L^2 the closest member lifts the heights to 0.2009, 0.2589, 0.2009, which opens small gaps at the stations and closes larger ones between them. Each is closer in the norm it minimizes. For the sines the two projections coincide, because the sines are orthogonal in both inner products.

Piecewise-linear interpolation as a hat-function expansion

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

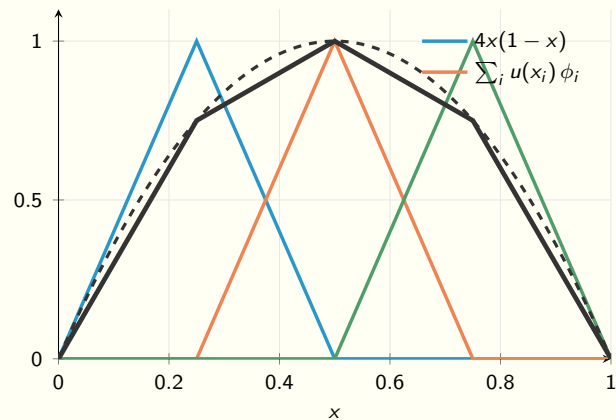
Noise and GPs

Families

Adaptive shapes

The fin

Hats to networks



$$u_h(x) = \sum_i u(x_i) \phi_i(x)$$

Look again at the straight segments. They are a basis too. Take one shape per interior station, equal to one at its own station, zero at the others, and straight in between, a hat function.

Weight each hat by the reading at its station and add. The sum is the straight-segment guess, $u_h(x) = \sum_i u(x_i) \phi_i(x)$, so the readings are the coefficients, and the energy projection found this same curve a moment ago.

Representing a kink in the hat basis and the sine basis

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

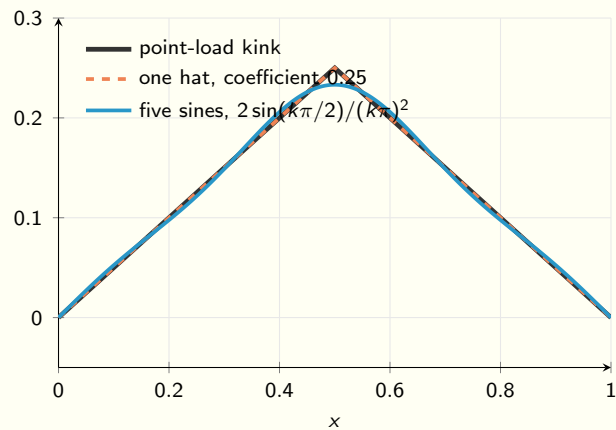
Noise and GPs

Families

Adaptive shapes

The fin

Hats to networks



one hat on the corner
one number, 0.25, no error

three hats, five stations
0.125, 0.25, 0.125, no error

sines, three and thirty-three modes
4.8 and 0.20 percent

[Try it live](#)

Back to the point load. Which shapes describe its corner with few numbers?

One hat whose node sits on the corner does it with one number, 0.25. On the five-station grid three hats do it with 0.125, 0.25, 0.125 and zero error.

The sines need the infinite list $c_k = 2 \sin(k\pi/2)/(k\pi)^2$, which falls like $1/k^2$ and leaves 4.8 percent after three modes and 0.20 percent after thirty-three. A shape that shares the corner needs one number where smooth shapes need infinitely many.

Decay rates of sine coefficients

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

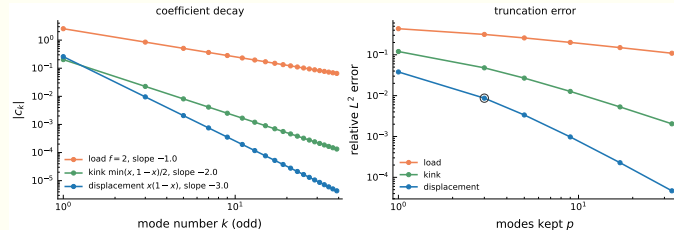
Noise and GPs

Families

Adaptive shapes

The fin

Hats to networks



load $f = 2$
 $c_k = 8/(k\pi)$, slope
 -1.0

kink
 $c_k = 2/(k\pi)^2$, slope
 -2.0

parabola
 $c_k = 8/(k\pi)^3$, slope
 -3.0

How fast do coefficients fall? Put three functions on the sine modes. The load $f = 2$ has $c_k = 8/(k\pi)$, the kink $2/(k\pi)^2$, and the parabola $8/(k\pi)^3$, since $-u'' = f$ divides the load's coefficients by $(k\pi)^2$.

A sine series represents the odd continuation of the function, and each derivative that continuation keeps continuous gains one power of k , which is what the slopes -1.0 , -2.0 , -3.0 show. One mode leaves 44 percent of the load and thirty-three still leave 11, because every sine vanishes at the ends where the load does not. What happens beside a jump, where no derivative is continuous?

The Gibbs phenomenon at a jump

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

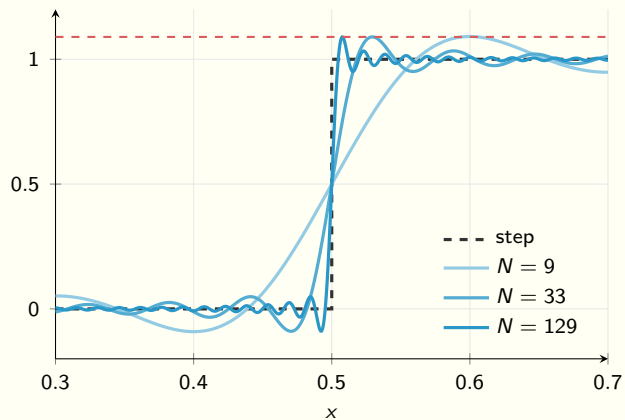
Noise and GPs

Families

Adaptive
shapes

The fin

Hats to
networks



overshoot, $N = 9$
0.091

overshoot, $N = 33$
0.090

overshoot, $N = 129$
0.089, **toward** 0.0895

Sum the sine series of a step. With nine modes the sum overshoots the top of the step beside the jump.

Add modes. The oscillation squeezes toward the jump, and the overshoot stays.

The overshoot tends to 0.0895 of the jump and never below it, so the series does not converge in the sup norm. It does converge in L^2 , because the region of large error narrows. That is the Gibbs phenomenon.

Conditioning of the Gram matrix for nearly parallel shapes

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

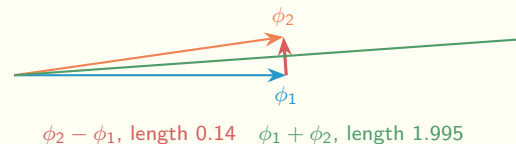
Noise and GPs

Families

Adaptive shapes

The fin

Hats to networks



$$\mathbf{G} = \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}, \quad \rho = 0.99, \quad \text{cond } \mathbf{G}$$

add $0.1(\phi_1 - \phi_2)$
target moves 0.014,
coefficients 0.1 and
-0.1

add $0.1(\phi_1 + \phi_2)$
target moves 0.2,
fourteen times more

$\sqrt{199}$
14.1

neighboring hats
 $\rho = 1/4$

five Gaussian bumps a
third of a width apart
cond $\mathbf{G} = 2.4$ million

So far the shapes were orthogonal. Now take a family whose shapes overlap. Two unit shapes with overlap $\rho = \langle \phi_1, \phi_2 \rangle = 0.99$ point almost the same way, and their Gram matrix has condition number $(1 + \rho)/(1 - \rho) = 199$. Where does that number show up?

Their difference has length $\sqrt{2 - 2\rho} = 0.14$, so adding $0.1(\phi_1 - \phi_2)$ to a target moves the target by 0.014 and the coefficients by 0.1 and -0.1 .

Adding $0.1(\phi_1 + \phi_2)$ moves the target by 0.2 for the same coefficient change, fourteen times more, and $14.1 = \sqrt{199}$. Neighboring hats overlap with $\rho = 1/4$, and five Gaussian bumps a third of a width apart reach a condition number of 2.4 million.

Approximating a corner with Bernstein polynomials

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

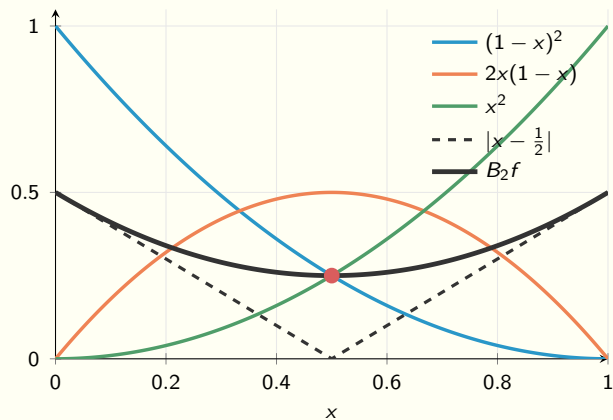
Noise and GPs

Families

Adaptive
shapes

The fin

Hats to
networks



the three hills at mid-span
 $\frac{1}{4}, \frac{1}{2}, \frac{1}{4}$

readings of $|x - \frac{1}{2}|$ at $0, \frac{1}{2}, 1$
 $\frac{1}{2}, 0, \frac{1}{2}$

B_2f at the corner
 $\frac{1}{4}$ **where** $f = 0$

Can a family of smooth shapes get as close as we like to a corner? Try polynomials. The three quadratic Bernstein shapes are three hills, one at each wall and one at mid-span with height $\frac{1}{2}$, and they add to one at every x .

Weight each hill by a reading of the corner function $|x - \frac{1}{2}|$ at $0, \frac{1}{2}, 1$, that is $\frac{1}{2}, 0, \frac{1}{2}$, and add. At mid-span the hills weigh $\frac{1}{4}, \frac{1}{2}, \frac{1}{4}$, so the sum reads $\frac{1}{4}$ where the function is 0. Every neighbor of the corner sits higher than the corner, so any average overshoots there, and the sum stays between the smallest and largest reading.

Convergence rate of Bernstein polynomials

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

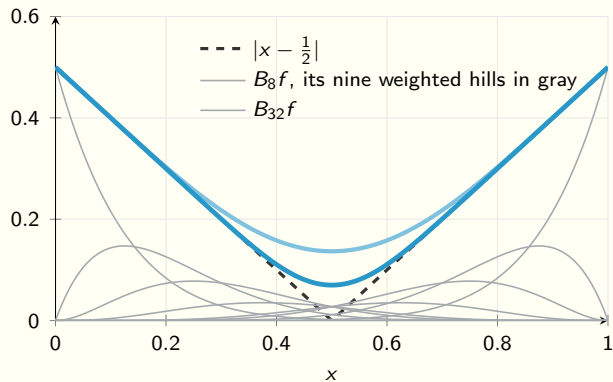
Noise and GPs

Families

Adaptive
shapes

The fin

Hats to
networks



$$B_n f(x) = \sum_{k=0}^n f\left(\frac{k}{n}\right) \binom{n}{k} x^k (1-x)^{n-k}$$

gap at the corner, $n = 2$
0.250

gap at the corner, $n = 8$
0.137, predicted
0.141

gap at the corner, $n = 32$
0.070, predicted
0.071

prediction
 $0.4/\sqrt{n}$

[Try it live](#)

Here is Bernstein's idea. Flip n coins that land heads with probability x , and weight the reading $f(k/n)$ by the probability of exactly k heads. Each hill is that probability, peaked at k/n . With eight coins the hills narrow and the sum closes in. The gap at the corner is the average distance of the fraction of heads from one half, about $0.4/\sqrt{n}$, which predicts 0.141 against the measured 0.137.

With thirty-two the gap is 0.070 against a prediction of 0.071. The hills narrow like $1/\sqrt{n}$ and so does the gap, and it always sits at the corner, where the function is least smooth.

Bernstein basis polynomials

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

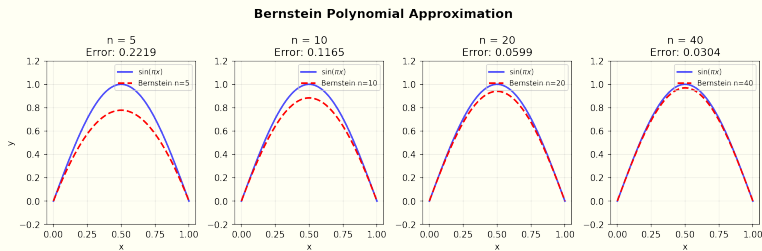
Noise and GPs

Families

Adaptive
shapes

The fin

Hats to
networks



$$B_{k,n}(x) = \binom{n}{k} x^k (1-x)^{n-k}, \quad B_n f(x) = \sum_{k=0}^n f\left(\frac{k}{n}\right) B_{k,n}(x)$$

largest gap,
 $n = 5, 10, 20, 40$
0.222, 0.117, 0.060,
0.030

the basis
**nonnegative, sums
to one, peaked at
 k/n**

Give Bernstein's construction the smooth target $\sin(\pi x)$. The basis shapes $B_{k,n}$ are nonnegative and add to one at every x , so $B_n f$ is a weighted average of the readings $f(k/n)$, and each shape peaks at its own reading's location k/n .

An average of readings of a concave function sits below it, so every $B_n f$ lies under the sine, and the gap at mid-span is largest. The largest gap is 0.222 at $n = 5$, 0.117 at $n = 10$, 0.060 at $n = 20$, and 0.030 at $n = 40$, so each doubling of n roughly halves it. The convergence is uniform, which is the Weierstrass theorem proved by construction.

Convergence of Bernstein polynomials for a smooth function

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

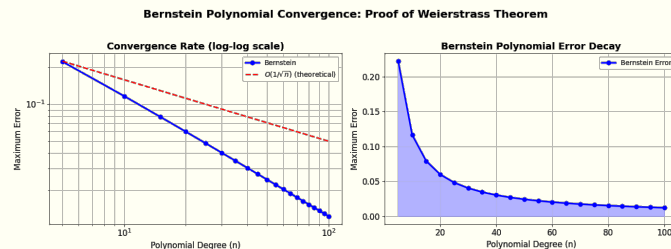
Noise and GPs

Families

Adaptive
shapes

The fin

Hats to
networks



target
 $\sin(\pi x)$

largest gap, $n = 5$ and
 $n = 100$
0.22 and 0.013

Voronovskaya at mid-span,
 $\pi^2/(8n)$
0.25 and 0.012

For the corner the gap fell like $1/\sqrt{n}$. Give Bernstein a smooth target, $\sin(\pi x)$, and the error falls faster, along a slope of -1 on the log-log plot, from 0.22 at $n = 5$ to 0.013 at $n = 100$.

Voronovskaya's formula predicts the gap for a twice-differentiable target, $f - B_n f \approx -x(1-x)f''(x)/(2n)$, which at mid-span has size $\pi^2/(8n)$, 0.25 at $n = 5$ and 0.012 at $n = 100$. So $1/\sqrt{n}$ is the rate for a corner, and one more derivative gives one more power of n , as it did for the sine coefficients.

Least-squares polynomial approximation of $\sin(\pi x)$

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

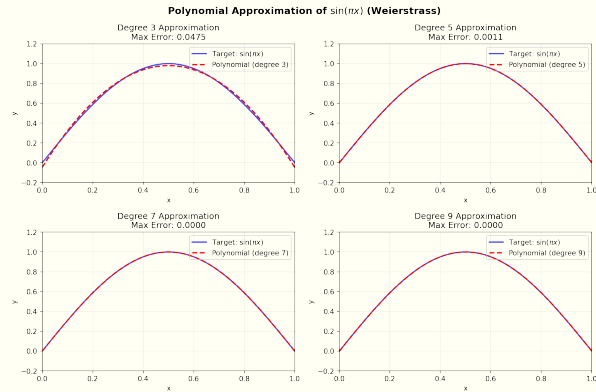
Noise and GPs

Families

Adaptive shapes

The fin

Hats to networks



degree 3, largest gap
0.0475

degree 5, largest gap
0.0011

degrees 7 and 9, largest gap
below 0.00005

Bernstein's polynomial is one polynomial of degree n , and a poor one. Fit $\sin(\pi x)$ by least squares on a hundred points instead. Degree 3 misses by 0.0475 in the sup norm, degree 5 by 0.0011, and degrees 7 and 9 by less than 0.00005.

For a function analytic on the interval, like the sine, the best polynomial error falls geometrically in the degree, while Bernstein's falls like $1/n$. The theorem promised only that some polynomial reaches any tolerance, and the least-squares fit reaches it at a far lower degree than Bernstein's construction.

Runge's phenomenon

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

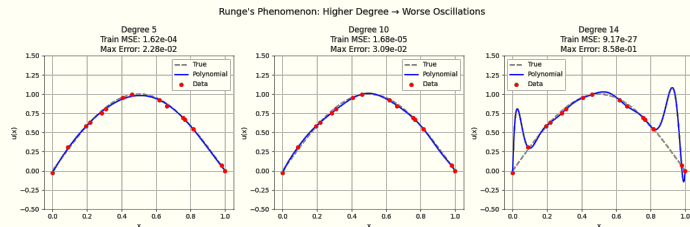
Noise and GPs

Families

Adaptive
shapes

The fin

Hats to
networks



degree 5, least squares
largest gap 0.023

degree 10, least squares
largest gap 0.031

degree 14, through all
fifteen readings
**training mean
squared error**
 9×10^{-27} , **largest
gap 0.86**

Now fit fifteen scattered readings of the same sine, by least squares at degrees 5 and 10, and with the degree-14 polynomial through all fifteen points. The two least-squares fits miss by 0.023 and 0.031 in the sup norm. The interpolant reproduces every reading to twenty-six digits and misses by 0.86 near the ends, where it swings between the data.

Between readings a high-degree polynomial is free to oscillate, and it does, most near the ends of the interval. That is Runge's phenomenon, and the degree-seven polynomial through the eight fin readings failed the same way, 31 percent against the truth.

The Weierstrass approximation theorem and its limits

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

Noise and GPs

Families

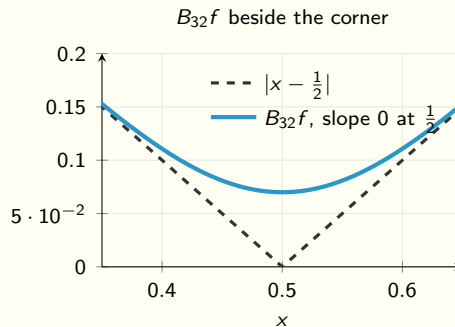
Adaptive shapes

The fin

Hats to networks

Weierstrass (1885)

For every continuous f on $[0, 1]$ and every tolerance $\epsilon > 0$ there is a polynomial p with $|f(x) - p(x)| < \epsilon$ at every point of the interval.



Bernstein's error
falls like $1/\sqrt{n}$

best degree- n polynomial
falls like $1/n$

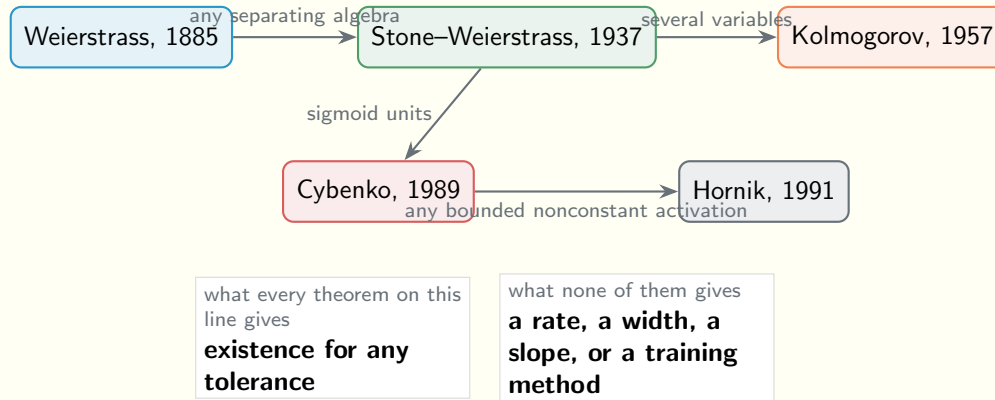
the theorem names
no polynomial, no degree, no slope

So polynomials come as close as we like to every continuous function. Notice what the theorem does not say. It names neither the polynomial nor its degree, and it says nothing about slopes, since every $B_n f$ has slope zero at the corner where the slope of f jumps from -1 to 1 . Bernstein's error falls like $1/\sqrt{n}$, the best degree- n polynomial does better, like $1/n$, and the theorem names neither.

Density is also different from having a closest member. Polynomials of a fixed degree contain their closest point, and the polynomials of all degrees have no closest member for the corner, since the distance can be made smaller than any positive number and is never zero.

Universal approximation theorems for neural networks have the same form and the same silences.

History of universal approximation theorems



Weierstrass in 1885 proved that polynomials come within any tolerance of every continuous function on an interval. Stone in 1937 extended the theorem to any algebra of continuous functions on a compact set that separates points and contains the constants, so the trigonometric polynomials and many other families are dense as well. Kolmogorov in 1957 wrote every continuous function of several variables as a finite sum of compositions of continuous functions of one variable, a two-layer form.

Cybenko in 1989 proved that networks with one hidden layer of sigmoid units are dense in the continuous functions on a box, and Hornik in 1991 showed that any bounded nonconstant activation gives the same result. Every theorem on this line has the form Weierstrass gave it, existence for any tolerance, with no rate, no width, and nothing about slopes or training.

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

Noise and GPs

Families

Adaptive shapes

The fin

Hats to networks

Cauchy sequences, completeness, and Hilbert spaces

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

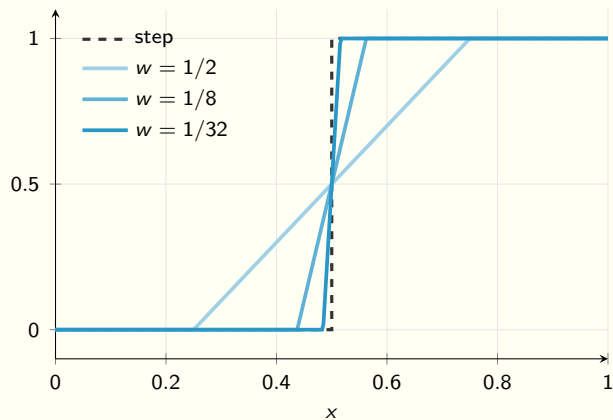
Noise and GPs

Families

Adaptive
shapes

The fin

Hats to
networks



L^2 distance to the step,
 $\sqrt{w/12}$
0.204 at $w = 1/2$

$w = 1/8$
0.102

$w = 1/32$
0.051

$\|f_8 - f_{32}\|_{L^2}$
0.077

A limit can lie outside a family in a second way. A continuous ramp of width w sits at L^2 distance $\sqrt{w/12}$ from the step.

Narrow the ramp. Every quartering of w halves the distance to the step.

The ramps also approach one another, $\|f_8 - f_{32}\| = 0.077$, a Cauchy sequence, and every ramp is continuous while the limit is the step. So the continuous functions with the L^2 distance are incomplete, and L^2 , where every Cauchy sequence converges, is their completion. A complete space with an inner product is a Hilbert space. The string's displacements form H_0^1 , the pinned functions of finite energy.

Point evaluation in L^2 and H^1

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

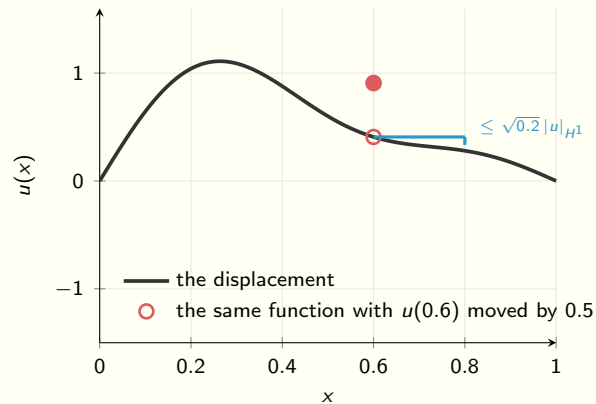
Noise and GPs

Families

Adaptive
shapes

The fin

Hats to
networks



L^2 distance between the two
0

$$|u(y) - u(x)| = \left| \int_x^y u'(s) ds \right| \leq \sqrt{y-x} |u|_{H^1}$$

A sensor reads a value at a point, and L^2 cannot. The plot shows the displacement and a second function equal to it everywhere except at $x = 0.6$, where it reads 0.5 higher. The two have L^2 distance zero, so in L^2 they are the same element, and an element of L^2 has no value at a point. One derivative restores it. For a finite-energy displacement, $|u(y) - u(x)| = \left| \int_x^y u'(s) ds \right| \leq \sqrt{y-x} |u|_{H^1}$, so values at nearby points differ by at most the square root of the separation times the slope norm, and every element of H^1 on an interval has one continuous representative. Our station readings were legitimate because the string had finite energy.

Now eight thermocouples on a cooling fin give eight such values, with noise, and nothing else. How do eight readings become a curve everywhere?

Kernels as similarity rules for scattered readings

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

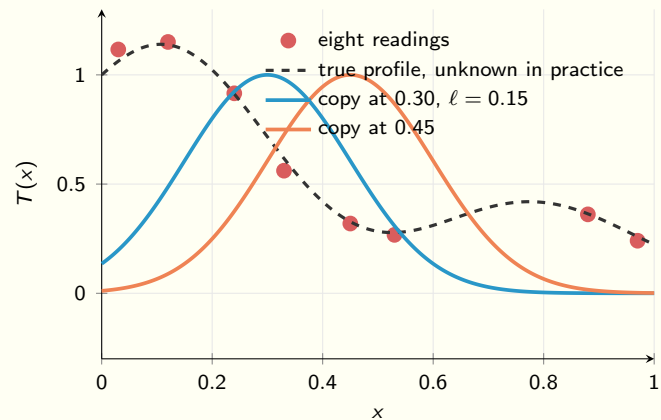
Noise and GPs

Families

Adaptive shapes

The fin

Hats to networks



$$k(x, x') = e^{-(x-x')^2/2\ell^2}$$

$$k(0.30, 0.45), \text{ one } \ell \text{ apart} \\ 0.61$$

$$k \text{ at } 3\ell \text{ apart} \\ 0.011$$

These are the eight readings from the fin. Infinitely many curves pass near them. What do we know that the readings do not say?

The profile is smooth, so nearby points should report similar values, and a reading should constrain the temperature for some distance along the fin.

A kernel says how far. $k(x, x') = \exp(-(x - x')^2/2\ell^2)$ assigns a similarity to every pair of locations. Its copies centered at 0.30 and 0.45 sit one length scale apart, and their similarity is $e^{-1/2} = 0.61$. Two locations 3ℓ apart have similarity 0.011, so a reading constrains the temperature for about two length scales.

Put one copy at each sensor, add them with weights, and choose the weights so the sum passes through every reading, $\mathbf{K}\alpha = \mathbf{y}$. Which weights does that solve produce?

Reconstructing a profile with weighted kernel copies

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

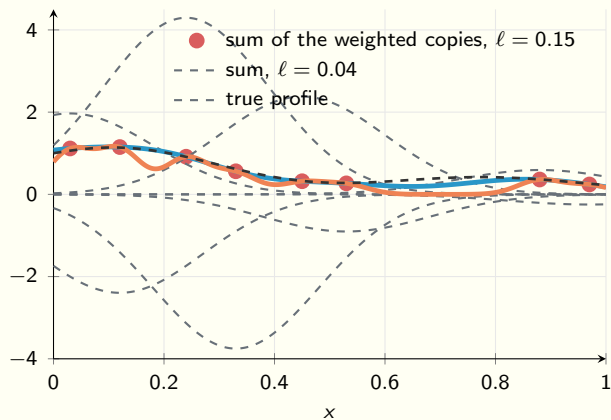
Noise and GPs

Families

Adaptive
shapes

The fin

Hats to
networks



$$\mathbf{K}\boldsymbol{\alpha} = \mathbf{y}, \quad K_{ij} = k(x_i, x_j)$$

weights at $\ell = 0.15$
**alternate in sign, up
to 4.30 and -3.75**

two equal readings one ℓ
apart
0.62 each

sum at $x = 0.7$, $\ell = 0.04$
**0.000 against the
true 0.393**

At $\ell = 0.15$ the eight copies overlap, and the solve gives them weights that alternate in sign, up to 4.30 and -3.75 . Two sensors a distance ℓ apart with equal readings get 0.62 each, because each neighbor already supplies part of what they share. The weighted copies sum to a curve that passes through every reading and stays smooth between them.

Shorten ℓ to 0.04 and each weight is nearly its own reading, so the sum collapses between sensors, to 0.000 at $x = 0.7$ where the true profile is 0.393. A similarity that reaches too short a distance leaves the curve unconstrained between sensors.

Positive definiteness of a kernel

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

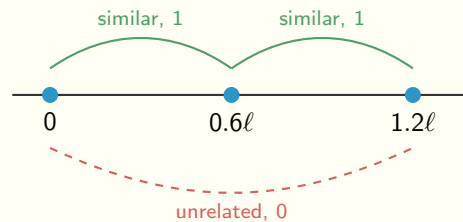
Noise and GPs

Families

Adaptive shapes

The fin

Hats to networks



$$\mathbf{w} = (1, -\sqrt{2}, 1), \quad \mathbf{w}^\top \mathbf{K} \mathbf{w} = -1.66$$

$$\mathbf{K} = \begin{pmatrix} 1 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 1 \end{pmatrix}, \quad \lambda = 1 - \sqrt{2}, 1, 1 + \sqrt{2}$$

$$k(x, x') = \phi(x)^\top \phi(x') \Rightarrow \mathbf{w}^\top \mathbf{K} \mathbf{w} = \left\| \sum_i w_i \phi(x_i) \right\|^2 \geq 0$$

Could we have picked any similarity rule? Try one. Put sensors at 0 , 0.6ℓ , 1.2ℓ , and call two of them similar, with value one, when closer than ℓ and unrelated otherwise. The matrix has eigenvalues $1 - \sqrt{2} = -0.414$, 1 , and $1 + \sqrt{2}$, and the weights $(1, -\sqrt{2}, 1)$ give $\mathbf{w}^\top \mathbf{K} \mathbf{w} = -1.66$. A negative value cannot be a squared length, so no inner product produces this rule.

A kernel is positive definite when $\mathbf{w}^\top \mathbf{K} \mathbf{w} \geq 0$ for every choice of sites and weights. Whenever $k(x, x') = \phi(x)^\top \phi(x')$ for some list of features, $\mathbf{w}^\top \mathbf{K} \mathbf{w} = \left\| \sum_i w_i \phi(x_i) \right\|^2 \geq 0$, and every positive definite kernel has this form with the list allowed to be infinite. So a kernel method is linear regression on features we never write down.

The reproducing kernel Hilbert space

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

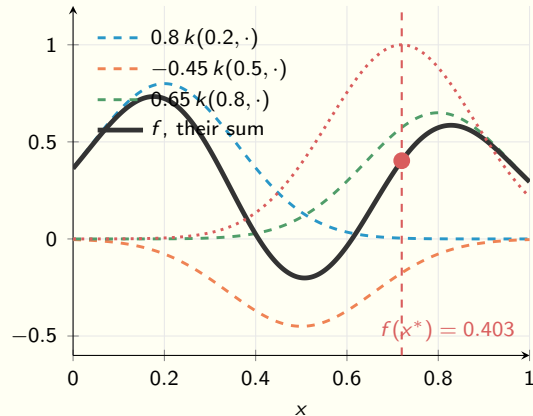
Noise and GPs

Families

Adaptive shapes

The fin

Hats to networks



$$\langle k(x_1, \cdot), k(x_2, \cdot) \rangle_k = k(x_1, x_2)$$

$$\|f\|_k^2 = \alpha^\top \mathbf{K} \alpha, \ell = 0.16$$

1.041

$$f(x) = \langle f, k(x, \cdot) \rangle_k$$

$$f(0.72), \text{ read as an inner product}$$

0.403, **bound 1.02**

$$\text{slope bound}$$

$$|f'(x)| \leq \|f\|_k / \ell$$

Now make the kernel's geometry explicit. Declare the inner product of two kernel copies to be their similarity, $\langle k(x_1, \cdot), k(x_2, \cdot) \rangle_k = k(x_1, x_2)$, extend it by linearity, and complete the collection. That is the reproducing kernel Hilbert space, and its overlap differs from the L^2 overlap, 0.61 against 0.78 for two unit Gaussian bumps a distance ℓ apart.

Take a member, three weighted copies with $\ell = 0.16$, $f = 0.8 k(0.2, \cdot) - 0.45 k(0.5, \cdot) + 0.65 k(0.8, \cdot)$. Its squared norm is $\alpha^\top \mathbf{K} \alpha = 1.041$, the similarities of the centers weighted by the coefficients.

Pair f with the copy centered at $x^* = 0.72$, the dotted curve. The rule returns $0.8 k(0.2, x^*) - 0.45 k(0.5, x^*) + 0.65 k(0.8, x^*)$, which is $f(x^*) = 0.403$, and in general $f(x) = \langle f, k(x, \cdot) \rangle_k$ for every member. Evaluating at a point is an inner product, continuous by Cauchy–Schwarz, $|f(x)| \leq \|f\|_k \sqrt{k(x, x)} = 1.02$ here, which is what L^2 could not give us. Differentiate the identity and the slope is bounded too, $|f'(x)| \leq \|f\|_k / \ell$, so a function that climbs by 1 over $\ell/10$ has norm at least 10. The norm measures roughness, and we are about to penalize it.

Noise amplification in kernel interpolation

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

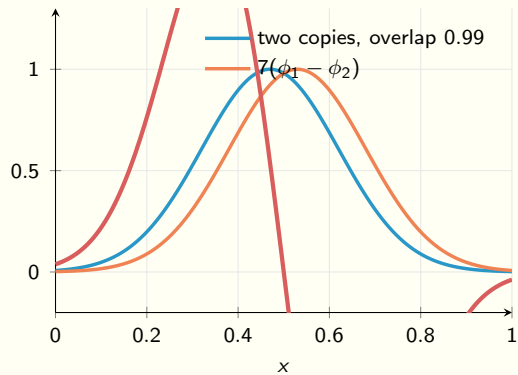
Noise and GPs

Families

Adaptive
shapes

The fin

Hats to
networks



eigenvalues of $\mathbf{K}$, overlap 0.99
1.99 along $(1, 1)$, 0.01 along $(1, -1)$

readings $(1, 1)$
weights near $\frac{1}{2}$ each

noise $(\varepsilon, -\varepsilon)$
weights $\pm\varepsilon/0.01$, a hundred times the noise

$\|\phi_1 - \phi_2\|$
0.14, so the curve moves fourteen times the noise

The interpolant reproduces every reading, noise included. How bad is that? At overlap 0.99 the two-sensor matrix has eigenvalues 1.99 and 0.01, along $(1, 1)$ and $(1, -1)$, and a reading pair $(1, 1)$ gives weights near one half each.

A noise pair $(\varepsilon, -\varepsilon)$, the difference between two nearly equal readings, gives weights $\pm\varepsilon/0.01$, a hundred times the noise. The hundredfold is in the coefficients along $\phi_1 - \phi_2$, whose function has norm 0.14, so the curve moves by fourteen times the noise while its coefficients move by a hundred.

We have changed the question. Approximating a known function asked which member is closest. Recovering an unknown function from noisy readings asks whether the readings identify it, and interpolation answers by trusting every reading to the last digit.

Kernel ridge regression and the choice of λ

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

Noise and GPs

Families

Adaptive
shapes

The fin

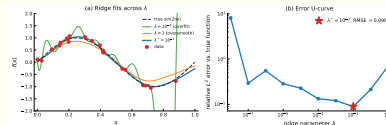
Hats to
networks

$$\min_{f \in \mathcal{H}_k} \sum_{i=1}^n (f(x_i) - y_i)^2 + \lambda \|f\|_k^2$$

$$(\mathbf{K} + \lambda \mathbf{I})\alpha = \mathbf{y}$$

$\lambda = 0.1$, small divisor
 $0.01 \rightarrow 0.11$

amplification
a hundred $\rightarrow$ nine



So add a roughness penalty to the misfit and minimize $\sum_i (f(x_i) - y_i)^2 + \lambda \|f\|_k^2$ over the whole space. Any component of f orthogonal to the span of the data's kernel copies changes no reading and raises the norm, so the minimizer lies in that span, and substituting gives $(\mathbf{K} + \lambda \mathbf{I})\alpha = \mathbf{y}$, the interpolation solve with the identity added to the diagonal. At $\lambda = 0.1$ the small divisor rises from 0.01 to 0.11 and the amplification drops from a hundred to nine.

The figure shows twenty noisy sine samples across nine decades of λ . Near-interpolation reaches ninety times the best error, the minimum sits near 10^{-1} , and over-smoothing rises beyond it. The matched weight $\lambda = \sigma^2 = 10^{-2}$ is one decade below this sample's minimum, and in practice we choose λ by predicting held-out readings.

Conditioning a Gaussian on an observation

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

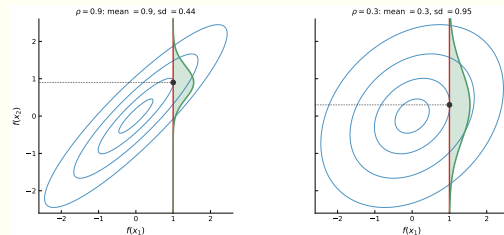
Noise and GPs

Families

Adaptive
shapes

The fin

Hats to
networks



$$f(x_1) = Z_1, \quad f(x_2) = \rho Z_1 + \sqrt{1 - \rho^2} Z_2$$

$$f(x_2) \mid f(x_1) = y \sim \mathcal{N}(\rho y, 1 - \rho^2)$$

$\rho = 0.9$
**mean $0.9y$, standard
deviation 0.44**

$\rho = 0.3$
**mean $0.3y$, standard
deviation 0.95**

The same computation reads as probability. Build two temperatures from independent standard normals, $f(x_1) = Z_1$ and $f(x_2) = \rho Z_1 + \sqrt{1 - \rho^2} Z_2$, so both have variance one and covariance ρ .

Observe $f(x_1) = y$. That fixes $Z_1 = y$, and $f(x_2) = \rho y + \sqrt{1 - \rho^2} Z_2$ is Gaussian with mean ρy and standard deviation $\sqrt{1 - \rho^2}$. At $\rho = 0.9$ the slice has mean 0.9 and standard deviation 0.44, at $\rho = 0.3$ it keeps mean 0.3 and standard deviation 0.95.

A Gaussian process makes any finite list of values jointly Gaussian with covariances $k(x_i, x_j)$, and this is the case $n = 2$.

Gaussian-process reconstruction of the fin profile

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

Noise and GPs

Families

Adaptive
shapes

The fin

Hats to
networks



Condition the prior on the eight noisy readings and every location gets a Gaussian. Its mean is $\mathbf{k}(x)^\top (\mathbf{K} + \sigma^2 \mathbf{I})^{-1} \mathbf{y}$, the ridge curve with $\lambda = \sigma^2$, and at $\ell = 0.15$ it is off by 10 percent. Its variance is $k(x, x) - \mathbf{k}(x)^\top (\mathbf{K} + \sigma^2 \mathbf{I})^{-1} \mathbf{k}(x)$, the prior's variance minus what the sensors explained. The two-standard-deviation band collapses at the sensors and swells to 0.90 in the unsensed stretch, where the nearest sensor correlates with $x = 0.7$ at only 0.53.

Let the readings choose ℓ by the marginal likelihood, the probability the model assigns to the data seen. They pick $\ell = 0.31$, where the mean is within 3.2 percent, the band in the gap is 0.12, and the value at $x = 0.7$ is 0.389 against the true 0.393. Remember what the band is. It is the model's own uncertainty, conditional on the kernel, the noise level, and the length scale.

Approximating a family of functions with a fixed basis

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

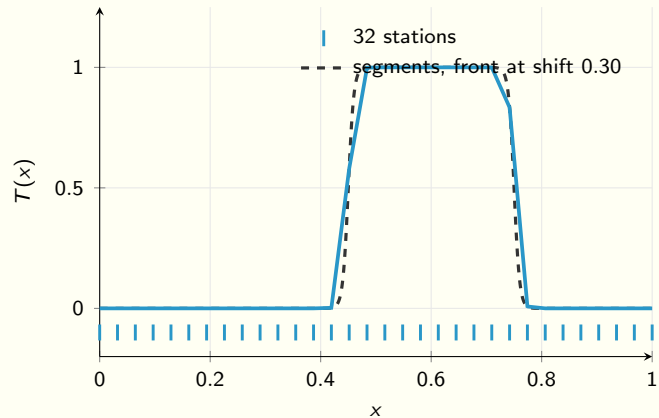
Noise and GPs

Families

Adaptive
shapes

The fin

Hats to
networks



32 stations, shift 0

8.7 percent

32 stations, shift 0.15

6.3 percent

32 stations, shift 0.30

9.6 percent

station spacing

**three times the edge
width**

numbers that rebuild the
front at every time

**two edges, a width,
an amplitude, a shift**

Now the target itself moves. Hot fluid pushes a temperature front along a pipe, one shape $T(x - ct)$ whose whole history is its position, and we want one description for every position at once.

Thirty-two stations have a spacing three times the edge width, and wherever the front sits most of them record a flat line, so the error is 8.7 percent at shift 0, 6.3 at 0.15, and 9.6 at 0.30. Two edge positions, a width, an amplitude, and a shift rebuild the front at every time, five numbers against thirty-two, once a number may mean where or how wide instead of how much at station x_i .

The Kolmogorov n -width

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

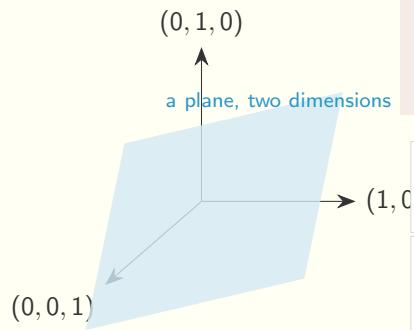
Noise and GPs

Families

Adaptive
shapes

The fin

Hats to
networks



Kolmogorov n -width

$$d_n(\mathcal{M}) = \min_{\dim V=n} \max_{f \in \mathcal{M}} \min_{v \in V} \|f - v\|$$

three unit spikes, squared
distances to any plane
sum to $3 - 2 = 1$

worst spike
at least
 $\sqrt{1/3} = 0.577$

How well can any fixed basis do on a whole family? Translate a unit spike across three grid points, so the snapshots are three perpendicular unit vectors. For any plane through the origin the three squared distances to it add up to $3 - 2 = 1$, so the worst snapshot sits at distance at least $\sqrt{1/3} = 0.577$, whichever plane we choose. A family that depends on one number needs all three dimensions.

The Kolmogorov n -width is the worst-case error of the best n -dimensional subspace. Read it from the inside out. The innermost minimum is the projection theorem, the maximum takes the hardest member, and the outer minimum designs the subspace with full knowledge of the family. No fixed basis of size n has a smaller worst-case error.

Widths of diffusion and transport families

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

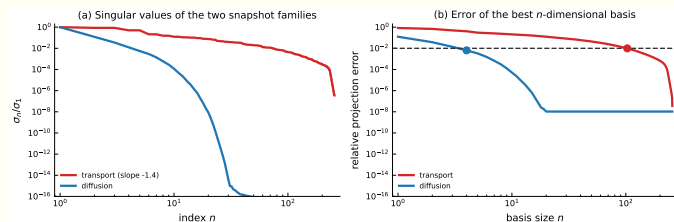
Noise and GPs

Families

Adaptive
shapes

The fin

Hats to
networks



diffusing pulse, one percent
four modes

translating pulse, one
percent
103 modes

coefficient k under
diffusion
shrinks by $e^{-(2\pi k)^2 t}$

coefficient k under
translation
changes phase only

Sample a family into a snapshot matrix, and its singular value decomposition gives the best subspace in the root-mean-square sense, a lower bound for the width.

Diffuse a steep-edged pulse and its k -th coefficient shrinks by $e^{-(2\pi k)^2 t}$, the high coefficients vanish, and four modes reach one percent. Translate the same pulse and the coefficient only changes phase, every frequency keeps its energy, and we need 103 modes.

Every translate of a sine lies in a two-dimensional span, so the difficulty is a sharp edge at every position, and the width says nothing about how few parameters the family depends on.

Adaptive representations with particles, splats, and selection

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

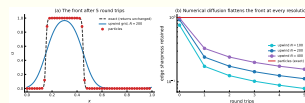
Noise and GPs

Families

Adaptive
shapes

The fin

Hats to
networks



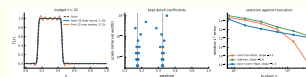
markers after five round
trips
front unchanged

upwind grid, edge
steepness kept
**eight to sixteen
percent**



sixteen trained Gaussians,
96 parameters
error 0.0125

fixed grid, 100 coefficients
error 0.30



thirty-two largest Haar
coefficients
1.3 percent

first thirty-two sines
5.1 percent

None of the three contradicts the width. Each drops its hypothesis of one fixed subspace.

Let the physics move the shapes. Markers ride the flow with their values, so the front returns after five round trips unchanged, while an upwind grid keeps eight to sixteen percent of the edge steepness, since each step averages neighbors.

Let an optimizer move them. Sixteen trained Gaussians, ninety-six parameters, reach 0.0125 on the two-ridge field against 0.30 for a fixed grid of one hundred coefficients, and the fit is non-convex with no guarantee.

Or keep the shapes and choose among them. The thirty-two largest Haar coefficients represent the front with 1.3 percent error against 5.1 for thirty-two sines, storing values and indices.

Choosing a representation for the cooling fin

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

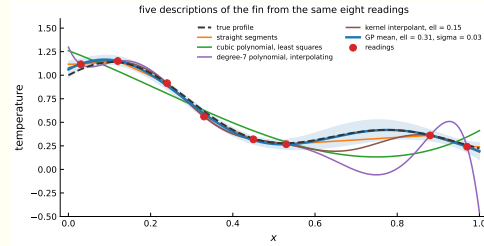
Noise and GPs

Families

Adaptive
shapes

The fin

Hats to
networks



description	error vs truth	held-out
straight segments	6.5 percent	0.095
cubic, least squares	23	0.310
degree-7 interpolation	31	3.57
kernel interpolant, $\ell = 0.15$	11	0.112
Gaussian-process mean, $\ell = 0.31$	3.2	0.034
noise level		0.03

Suppose we did not know the profile. What would still tell us which description to trust?

The noise level rules out the three exact interpolants. Withholding each reading and predicting it from the other seven ranks the five the same way, with the Gaussian-process mean near the noise level a perfect predictor would score. The physics admits the smooth family.

	Stored numbers, shape placement, and assumption for each family
Representation	
Stations	
Coefficients	
Norms	
Projection	
Bases	
Convergence	
Kernels	
Noise and GPs	
Families	
Adaptive shapes	
The fin	
Hats to networks	

Look back at every description we built, and ask what it stores, who places its shapes, and what it assumes about the field.

A family applies where its assumption is true, and when a fit goes wrong that assumption is the first thing to check. For stations it is aliasing, for sines the overshoot beside a jump, for kernels a length scale wrong for the field, and for splats a descent that stalls.

family	stores	shapes placed by	assumes
stations	values at fixed points	the analyst	linear variation between stations
hats	nodal values	the analyst and a mesher	piecewise-linear variation
sine modes	global coefficients	the analyst	a smooth odd periodic continuation
kernels and Gaussian processes	weights at the data sites	the data	smoothness at a length scale
particles	positions and carried values	the dynamics	values move with their carriers
Gaussian splats	centers, covariances, amplitudes	the optimizer	a smooth field with concentrated features
neural fields	network weights	the optimizer	the network's smoothness bias

Piecewise-linear interpolation as a one-hidden-layer ReLU network

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

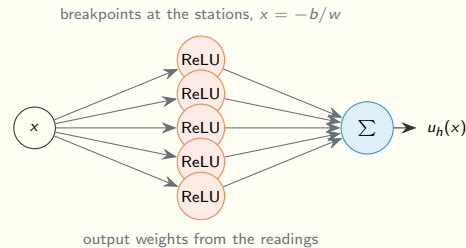
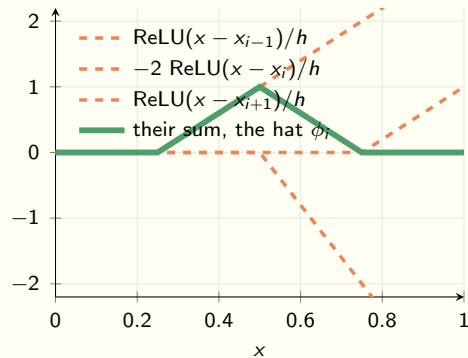
Noise and GPs

Families

Adaptive shapes

The fin

Hats to networks



bound $Mh^2/8$ at five stations

0.0156, met with equality by the parabola

[Try it live](#)

One last look at the hat. The first ramp, $\text{ReLU}(x - x_{i-1})/h$, starts at the station to the left. Subtract twice the ramp that starts at x_i . Add the ramp that starts at x_{i+1} .

The three ramps sum to the hat. So the straight-segment guess, a sum of hats weighted by readings, is a network with one hidden layer of ReLU units whose breakpoints sit at the stations. Its bound $Mh^2/8$ is a universal approximation theorem with a rate, and the parabola meets it with equality, 0.0156 at five stations.

Movable ReLU breakpoints on a traveling front

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

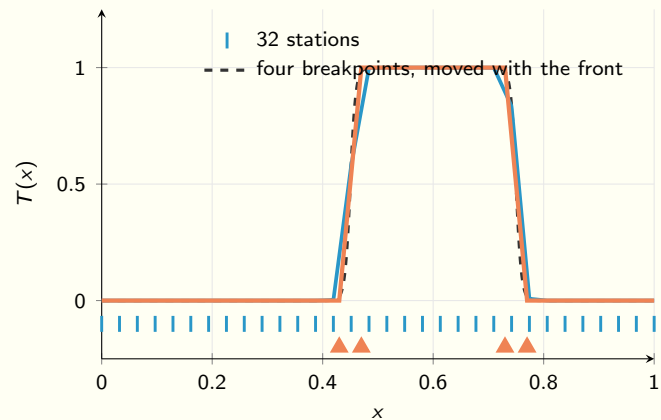
Noise and GPs

Families

Adaptive shapes

The fin

Hats to networks



breakpoint of
 $\text{ReLU}(wx + b)$
 $x = -b/w$

32 stations, front shifted
by 0.3

9.6 percent

four breakpoints, moved
with the front

4.9 percent

[Try it live](#)

A unit $\text{ReLU}(wx + b)$ has its breakpoint at $x = -b/w$, so the breakpoints are parameters we can place. Put four of them 0.02 on either side of each edge. They rebuild the front at 4.9 percent, against 8.7 for thirty-two stations.

Now move the front. The stations stay where we put them and their error rises to 9.6 percent, while the four breakpoints follow the edges and keep 4.9. The breakpoints use four numbers where the stations use thirty-two.

Polynomial and neural network approximation of $\sin(\pi x)$

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

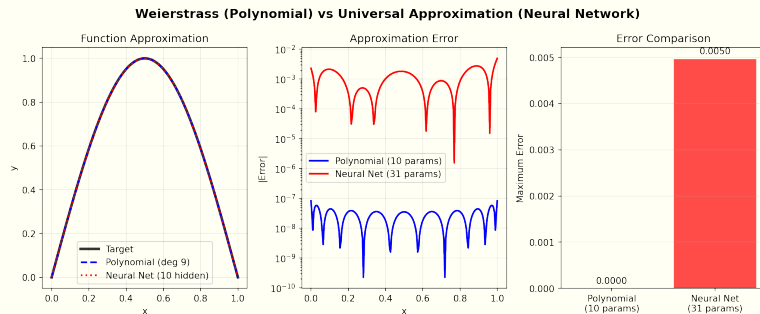
Noise and GPs

Families

Adaptive
shapes

The fin

Hats to
networks



polynomial, ten numbers
largest gap below
 10^{-7}

network, thirty-one
numbers
largest gap 0.0050

Put the two families on the same smooth target. A degree-nine polynomial fit by least squares, ten numbers, misses $\sin(\pi x)$ by less than 10^{-7} . A network with ten sigmoid units in one hidden layer, thirty-one numbers trained by Adam for ten thousand steps, misses by 0.0050.

On a smooth target on a fixed interval the polynomial's error is four orders of magnitude smaller, and the network's parameters go to placing breakpoints the target never needed. A polynomial is a sum of global shapes with fixed placement, it oscillates near the ends of the interval, and the number of its coefficients grows fast with the dimension. A network is a sum of local ramps with movable breakpoints, composed across layers, and its advantage lies where the polynomial fails, at concentrated or moving features, at scattered data, and in many dimensions.

Polynomials and neural networks compared

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

Noise and GPs

Families

Adaptive
shapes

The fin

Hats to
networks

	polynomials	one-hidden-layer networks
shapes	global, every coefficient changes the curve everywhere	local ramps, each unit changes the curve beyond its breakpoint
placement	fixed by the degree	breakpoints are trained parameters
failure on sparse data	Runge oscillations near the ends	a descent that stalls
growth with dimension	coefficients grow like $\binom{n+d}{d}$ in d dimensions	parameters set by width and depth
structure	one sum	layers compose

Put the two families side by side. A polynomial's shapes are global, so changing one coefficient changes the curve everywhere, while a ReLU unit changes the curve only beyond its breakpoint. A polynomial's shapes are fixed by the degree, while a network's breakpoints are trained. On sparse data a high-degree polynomial oscillates near the ends of the interval, and a network fails in a different way, when gradient descent stalls. A polynomial of degree n in d variables has $\binom{n+d}{d}$ coefficients, while a network's parameter count is set by its width and depth. And networks compose layers, which a single polynomial sum does not.

Shallow and deep networks on a high-frequency function

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

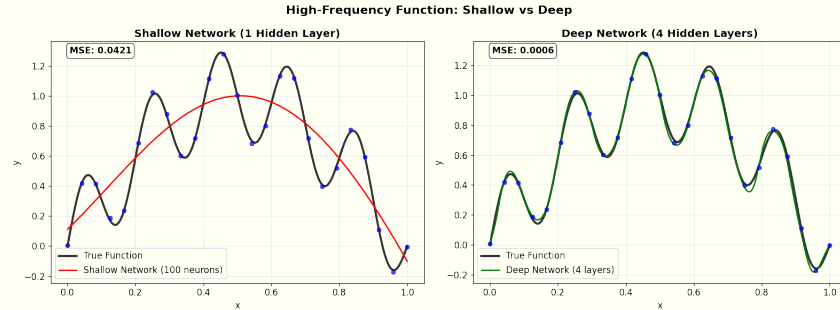
Noise and GPs

Families

Adaptive
shapes

The fin

Hats to
networks



one hidden layer, a
hundred units
mean squared error
0.0421

four hidden layers
mean squared error
0.0006

This target oscillates about five times across the interval, and both networks are trained by gradient descent on the blue readings. A one-hidden-layer network with a hundred units settles on a single broad arch, with mean squared error 0.0421, and a four-hidden-layer network follows every oscillation, with 0.0006.

Universality promised a shallow network with enough units. Training did not find it on this target, and depth did, and what depth adds is the next lecture's subject.

From fixed shapes to trainable breakpoints

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

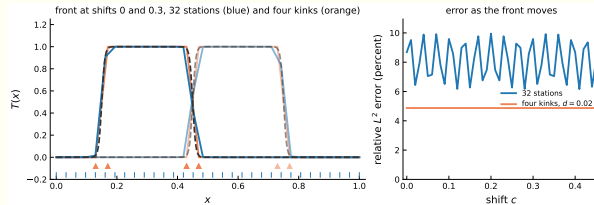
Noise and GPs

Families

Adaptive
shapes

The fin

Hats to
networks



fixed breakpoints
**a linear solve, with
the projection
theorem's guarantee**

moving breakpoints
**gradient descent,
with no guarantee**

A perceptron chooses between two problems. Fixed breakpoints give a linear problem, one solve, and the projection theorem's guarantee that the solve finds the closest member of the family. Moving breakpoints give a non-convex problem, gradient descent, and no such guarantee, the choice the splats already made. The gain is on fields whose features sit in a few places, the loss is the guarantee, and the width of a family says how much a fixed basis had to lose. The next lecture builds the perceptron from this unit and trains the breakpoints. Does universality survive other activations and more dimensions, and with which silences?

Summary

Representation

Stations

Coefficients

Norms

Projection

Bases

Convergence

Kernels

Noise and GPs

Families

Adaptive
shapes

The fin

Hats to
networks

- 1 Readings at fixed stations
- 2 Coefficients on shapes
- 3 Norms and the meaning of error
- 4 Inner products, projection, and orthogonality
- 5 Changing the basis
- 6 Convergence, density, and completeness
- 7 Kernels and point evaluation
- 8 Noisy observations, ridge regression, and Gaussian conditioning
- 9 Approximating a family of functions
- 10 Adaptive representations
- 11 Choosing a representation for the cooling fin
- 12 From hats to a network

We stored a function as a list of numbers and a rule, readings joined by segments or coefficients on shapes, and a fixed family was exact on its members and approximate outside them. We measured error in a norm the physics chose, and two functions could be close in one norm and far apart in another without bound. We found the closest member of a family by making the error orthogonal to the family, one linear solve, and saw that the inner product was a choice. Coefficients on a basis fell at a rate set by the smoothness of the function relative to the shapes, and a jump left an overshoot that never converged away in the sup norm. Weierstrass gave density with no rate, and existence of a closest member needed a complete space and a closed family. L^2 had no point values and H^1 restored them in one dimension, so kernels could turn scattered readings into a curve with a norm that measures roughness and a band conditional on the model. A family of functions had a width that bounded every fixed basis at once, and particles, trained shapes, and selection dropped the fixed-basis hypothesis in three ways. A hat is three ReLU ramps, so the straight-segment guess was already a network, and training moves its breakpoints.